

AI SHOPPING ASSISTANT CHATBOT

Project Documentation & Solution Report

Python | FastAPI | SQLite | SQLAlchemy | OpenAI Tool Calling

1. Executive Summary

The AI Shopping Assistant Chatbot is a full-stack Python e-commerce application that provides conversational product discovery and assistance. Users can search products, compare options, ask specification questions, and receive personalized recommendations.

The key design principle is grounding: the language model is instructed not to invent product information. Product-related information is retrieved from SQLite through Python functions exposed as OpenAI tools, and the returned catalog data is then supplied to the model for the final response.

2. Original Assignment Requirements

Intent classification: search, compare, specification-question, and recommendation.

Product-data grounding: query the product database and ground answers in actual catalog data.

Multi-turn memory: carry context across messages.

Comparison: generate side-by-side feature summaries.

Personalization: use browsing/purchase history.

Clarification: ask follow-up questions when important information is missing.

3. Implemented Solution

The submitted implementation uses an LLM with native function/tool calling. The LLM performs natural-language understanding and chooses database-backed functions. The four main tools are `search_catalog`, `compare_products`, `get_product_specs`, and `get_recommendations`.

`search_catalog` supports query/category/min/max price. `compare_products` returns prices and specifications for two products. `get_product_specs` returns product details. `get_recommendations` uses browsing and purchase history to identify preferred categories and brands.

4. System Architecture

User → Browser chatbot → POST /api/chat → FastAPI → ChatSession history → OpenAI model → database-backed tool → SQLite/SQLAlchemy → tool result → LLM → grounded response → ChatSession update → Browser.

The grounding boundary is between the LLM and the product database: factual product information is obtained through application tools rather than invented from model knowledge.

5. Detailed Working Flow

1) The browser sends message, `user_id` and `session_id`. 2) FastAPI retrieves or creates ChatSession. 3) Previous history is reconstructed. 4) The LLM selects a tool when catalog information is required. 5) Python queries SQLite. 6) Tool output is returned to the LLM. 7) The LLM generates the final response. 8) User and assistant messages are persisted.

6. Intent-to-Tool Mapping

Search → `search_catalog` → name, description, category, price.

Compare → `compare_products` → product name, price, specifications.

Specification → `get_product_specs` → name, price, description, specifications.

Recommendation → `get_recommendations` → browsing history, purchase history, category, brand.

7. Product Catalog and Database

The application uses SQLite through SQLAlchemy ORM. Product stores id, name, category, brand, price, description, specifications and image_url. Specifications are JSON.

The seed data contains 50 explicitly defined products across 12 named categories, followed by 200 generated synthetic products, for an intended total of approximately 250 products.

8. Personalization

Browsing history is updated when a logged-in user opens a product page; the latest 20 viewed product IDs are retained. Purchase history is updated by checkout. Recommendations match categories or brands represented in history and exclude history items.

9. Clarifying Questions

Clarification is prompt-guided. The system prompt tells the assistant to ask a polite follow-up question when a request is vague, such as asking for budget or intended use when the user only says 'Recommend a laptop'.

10. Grounding and Hallucination Prevention

The system prompt explicitly prohibits invented prices, specifications and products. The assistant has database-backed tools, tool outputs are passed to the model before final generation, and missing products produce not-found results instead of fabricated data.

For evaluation, verify that generated prices/specifications exactly match database records.

11. Frontend and User Experience

The project includes a Jinja2/HTML/CSS/JavaScript storefront, category filtering, live search, product details, login/register, cart using localStorage, checkout simulation, dashboard, and chatbot widget.

12. Project Structure

main.py – FastAPI routes and chat endpoint; ai_assistant.py – LLM tools and prompt; database.py – SQLite/SQLAlchemy connection; models.py – ORM models; seed_data.py – catalog; generate_products.py – synthetic data; reset_db.py – reseeding; static/ – frontend JS/CSS; templates/ – HTML pages.

13. Installation and Execution

Install Python 3.8+, create/activate a virtual environment, run `pip install -r requirements.txt`, create .env with OPENAI_API_KEY, run `python reset_db.py`, then `uvicorn main:app --reload` and open `http://localhost:8000`.

14. Evaluation Test Plan

Test search with a budget; ask a specification question; compare two known products; request recommendations after browsing/purchase activity; test a follow-up question; test an ambiguous request; test a nonexistent product; test a no-match search.

15. Evaluation Rubric Mapping

Grounding & Accuracy (30%): strong alignment through database-backed tools and anti-hallucination prompt.

Conversational Flow (20%): ChatSession JSON history and multi-turn reconstruction.

Recommendation Relevance (20%): browsing/purchase history mapped to category/brand.

Handling Ambiguity (15%): prompt-guided clarifying questions.

Code Quality & Documentation (15%): modular separation plus project documentation.

16. Strengths

Clear LLM/database separation; grounded product facts; persisted conversation history; actual browsing and purchase signals; search/compare/spec/recommendation tools; complete demonstrable e-commerce UI.

17. Current Limitations and Improvements

Intent is implicit through LLM tool selection rather than a dedicated classifier. Clarification is prompt-based. Recommendation ranking is simple and partly random. Browser-generated session IDs should be replaced with robust server-side sessions/UUIDs. Authentication cookies should be signed/managed securely. The README says gpt-4o/gpt-3.5-turbo while current ai_assistant.py calls gpt-5-nano. The README's 'mock payment gateway' is better described as checkout simulation. Synthetic catalog data should be replaced with maintained real catalog data in production.

18. Demonstration Script

Demonstrate budget search, a follow-up specification question, comparison, login and browsing, checkout simulation, personalized recommendations, ambiguous-request clarification, and a nonexistent-product query to show grounding.

19. Conclusion

The submitted project is a practical solution to the assigned AI Shopping Assistant problem. Its strongest feature is the grounded tool-calling architecture, supported by persistent conversation memory and basic personalization. Testing and demonstrating grounding, multi-turn context, comparisons, recommendations and clarification behavior should be the focus of the final evaluation.